

SGLang 前缀缓存测量 与 /generate 请求流程分析

第二次挑战 · 实验报告

模型 Qwen/Qwen3-0.6B SGLang 0.5.14 Ray 2.56.0
平台 WSL2 Ubuntu 26.04 Intel Core Ultra 5 338H (12 vCPU, 纯 CPU 推理)
学号 0102603133

一、摘要

本次挑战在第一次挑战已部署好的 SGLang 0.5.14 在线服务上，测量了 **Radix Cache 前缀复用** 的实际效果。两组负载各 32 条请求、最大并发 8，输入长度同为 2112 token，采样参数完全一致，唯一差别是前缀能否被复用。结果显示：共享前缀组把 **97.01%** 的 prompt token 变成缓存命中，实际执行 Prefill 的 token 数由 67,584 降至 2,019（降幅 97.0%），TTFT 的 p50 由 37,520.4 ms 降至 17,338.0 ms，端到端 p50 由 66,321.2 ms 降至 29,950.0 ms，输出吞吐由 1.92 提升到 4.31 tok/s。TPOT 表面上也从 1,976.36 ms 降到 864.03 ms，但那是「同批同步」下与 TTFT 此消彼长的结果，**不能**读作 Decode 变快（见 6.1）。两组成功率均为 32/32。

这与因果自注意力下「K/V 只由该位置及其之前的 token 决定」的性质一致：前缀缓存削减的是 **Prefill 的计算量**，而不改变 Decode 每步需要读取的上下文长度（上下文长 N 与步数两组完全一致）。把两组多出的 Prefill token 按服务端实测的 Prefill 吞吐折算，恰好能解释端到端时长的全部差异（见 6.2）。任务二沿指定主线逐跳核对了 v0.5.14 源码，给出了覆盖「请求接收 → 等待队列 → 前缀匹配 → Prefill/Decode → 缓存写入 → 流式输出」的流程图与说明。

二、实验环境与软件版本

项	值
宿主 / 发行版	Windows + WSL2 · Ubuntu 26.04 LTS (Resolute Raccoon)
CPU	Intel Core Ultra 5 338H (代号 Panther Lake) , 12 vCPU; 无 AVX-512、无 AMX
GPU / NPU	Intel Arc B370 核显 + NPU。未参与计算：WSL 内无 i915/xe DRM 驱动，缺 Intel 计算运行时（Level-Zero / OpenCL / SYCL），/dev/dri 仅由内核 vgem 虚拟桩提供
Python	3.12.14 (standalone, 与发行版自带 3.14 解耦)
SGLang	0.5.14 (源码构建, --device cpu)
Ray	2.56.0 (独立 venv, 与 SGLang 通过 HTTP 通信)
模型	Qwen/Qwen3-0.6B (bfloat16)
Radix Cache	开启 (未传 --disable-radix-cache; 服务端日志有 Tree cache initialized ... RadixCache)

服务端自报的缓存与批调度配置 (摘自服务端启动日志, 完整日志见 version_info.log)

disable_radix_cache	False (False = 启用 Radix Cache)
radix_cache_backend	None
radix_eviction_policy	lru
page_size (命中粒度对齐)	1
chunked_prefill_size	2048
max_prefill_tokens	16384
schedule_policy	fcfs
attention_backend	torch_native
disable_chunked_prefix_cache	False
device	cpu
dtype	auto
tp_size	1
KV 实际分配	[2026-09-29 10:56:39] KV Cache is allocated. dtype: torch.bfloat16, #tokens: 82206, K size: 4.39 GB, V size: 4.39 GB
运行上限 (服务端自报)	[2026-09-29 10:56:39] max_total_num_tokens=82206, chunked_prefill_size=2048, max_prefill_tokens=16384, max_running_requests=2048, context_len=40960, available_cpu_mem=2.24 GB
基数树初始化	[2026-09-29 10:56:39] Tree cache initialized: source=default impl=RadixCache hybrid_swa=False hybrid_ssm=False hierarchical=False streaming_wrapped=False

三、任务一：测量前缀缓存

3.1 两组负载的结构

负载	输入结构	测量前准备
共享前缀	2048 个 token 的共享前缀 + 64 个 token 的独立后缀 (合计 2112)	发送 1 条包含共享前缀的预热请求 (不计入结果)
分散前缀	输入长度均为 2112 个 token, 且首个 token 各不相同	不进行前缀预热

3.2 控制变量：如何做到「除前缀能否复用外，其余一致」

若两组各照字面自造一份语料，内容差异会与「前缀可否复用」混在一起。为把自变量隔离出来，分散前缀组按下面的方式构造（记号： $SHARED = base[0:2048]$ ， $SUFFIX[i] = base[2048+64i : 2112+64i]$ ， u_i 为 32 个两两互异的合法 token id）：

共享组第 i 条 = $SHARED[0:2048] + SUFFIX[i]$
分散组第 i 条 = $[u_i] + SHARED[1:2048] + SUFFIX[i]$

于是两组：输入长度同为 2112；独立后缀逐 token 相同；承载共享前缀的 2048 个 token 中有 **2047 个完全相同**；唯一差别是位置 0 的 token 两两不同。而 RadixCache 是从根节点按 token 逐位匹配的，位置 0 一旦不同，两条序列在树中就没有公共可达节点，前缀复用率为 0 —— 自变量因此被隔离为单一变量。构造逻辑与断言见 `src/target1/prefix_cache_bench.py` 的 `build_workloads()`。

3.3 测量流程

- 服务预热：首次实验前先发一条 **短请求**（16 token）完成预热，结果丢弃。
- 每组测量前：等待已有请求全部结束 → `POST /flush_cache` 并确认返回 `Cache flushed.`（HTTP 200）。
- 共享前缀组在 `flush` 之后再发 **1 条包含共享前缀的预热请求**（2048+64），结果丢弃；分散前缀组不做任何预热。
- 按序号顺序提交 32 条测量请求，**最大并发 8**，全部使用原生 `/generate + input_ids + stream=true`。
- 逐请求从服务端返回的 `meta_info` 读取 `prompt_tokens` 与 `cached_tokens`；TTFT 取「首个携带生成 token 的分片到达」的时刻， $TPOT = (\text{端到端} - TTFT) / (\text{生成 token 数} - 1)$ 。

指标定义（与任务书一致）

缓存命中率 = $\Sigma \text{cached_tokens} / \Sigma \text{prompt_tokens}$ ；实际 Prefill token 数 = $\Sigma(\text{prompt_tokens} - \text{cached_tokens})$ 。
分位数 p50 / p95 采用线性插值（与 numpy 默认方法一致）。吞吐量 = $\Sigma \text{completion_tokens} / \text{本组墙钟时间}$ 。

四、任务一：测量结果 (run-1)

4.1 主对照表

指标	共享前缀 shared_prefix	分散前缀 dispersed_prefix	说明
成功率	32/32 (100.0%)	32/32 (100.0%)	两组测量请求均全部成功
吞吐量 (输出 tok/s)	4.31	1.92	Σ completion_tokens / 墙钟
吞吐量 (请求 req/s)	0.269	0.120	32 / 墙钟
缓存命中率	97.01%	0.00%	Σ cached / Σ prompt, 取自服务端 meta_info
实际 Prefill token 数	2,019	67,584	Σ (prompt - cached); 总 prompt 各 67,584 / 67,584
TTFT p50 (ms)	17,338.0	37,520.4	首 token 到达时间
TTFT p95 (ms)	17,949.4	56,424.9	p95 反映高并发下的长尾
TPOT p50 (ms)	864.03	1,976.36	
TPOT p95 (ms)	1,832.94	3,595.13	输出 token 间平均间隔
端到端 p50 (ms)	29,950.0	66,321.2	整条请求耗时
端到端 p95 (ms)	30,618.5	69,381.4	
墙钟时间 (s)	118.9	266.2	并发 8 下 32 条请求的总耗时

数据源: results/target1/{shared_prefix,dispersed_prefix}/run-1/summary.json; 逐请求明细见同目录 requests.json1。

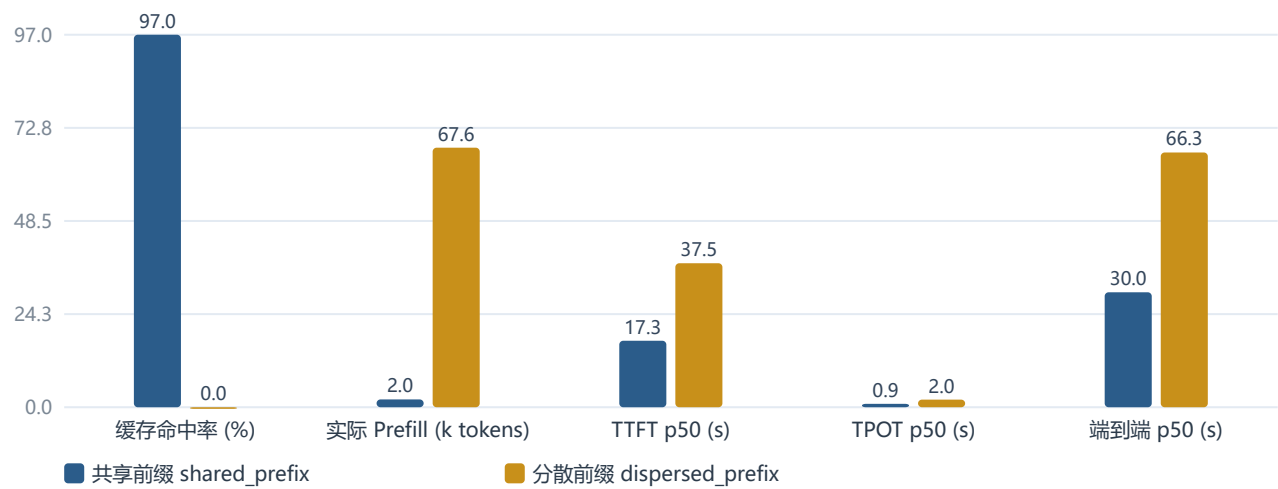


图 1 两组关键指标对照 (不同量纲, 按各自数值绘制, 用于观察差异方向与量级)。

4.2 服务端侧的独立旁证

组	服务端 Prefill 批次总数	其中 #cached-token > 0	本组服务端累计 #new-token	本组服务端累计 #cached-token
共享前缀	8	8	2,019	65,565
分散前缀	36	0	67,584	0

上表由 server_log_slice.log 中的 Prefill batch, #new-token: ..., #cached-token: ... 行直接汇总, 与服务端自报数据独立于客户端统计, 可与主表互相印证。

五、任务一：从因果自注意力公式解释前缀复用

单头缩放的因果自注意力为

$$\text{Attention}(Q, K, V) = \text{softmax} \frac{Q K^T}{\sqrt{d_k}} V$$

其中 $Q = XW_Q$, $K = XW_K$, $V = XW_V$, X 是长度 N 的输入隐状态序列。加入因果掩码 M 后

$$\text{Attention} = \text{softmax} \frac{Q K^T}{\sqrt{d_k}} + M V, \quad M_{ij} = 0 (j \leq i), \quad M_{ij} = -\infty (j > i)$$

掩码的作用是让位置 t 的输出只能注意到 $j \leq t$ 的键值，即注意力权重的幅值在 $j > t$ 处被置零。

5.1 为什么相同 token 与位置的前缀可以复用 K/V

关键是 K_i 与 V_i 只由位置 i 的隐状态决定：

$$K_i = h_i W_K, \quad V_i = h_i W_V$$

而 h_i 又逐层只由前 i 个 token 的表示算出。因果掩码只限制「能看多远」，**不会改变已经算出来的某个位置的 K/V 数值**——它作用在 qk^T 上，而不是作用在 K 、 V 本身。因此，若两个请求的 token 序列在位置 $0 \dots L-1$ 上逐位相同（token 相同、位置相同，且模型权重与 dtype 相同），那么这 L 个位置在每一层的 K/V **在数值上完全一致**，缓存它们就是安全的：

- 把先前请求在这 L 个位置已经算好、并已按 paged 布局写进 KV Cache 的 token $\rightarrow$ slot 索引保留在 RadixCache 里；
- 新请求到达时，沿基数树逐 token 匹配得到前 L 个 token 的 slot 列表，写入 req.prefix_indices，并在组批时令 extend_input_len = $N - L$ ；
- 于是本轮 Prefill 只需对位置 $L \dots N-1$ 计算并写出 K/V 。

本实验的对应关系：共享前缀组的 $L \approx 2048$ （36 条请求共享的前缀），分散前缀组因位置 0 各不相同，匹配在根节点即失败， $L = 0$ 。

5.2 命中之后仍需计算哪些 token

1. **未命中后缀的全部 token**（位置 $L \dots N-1$ ）：它们的 Q 、 K 、 V 都必须新算并写出，本实验中即每条请求自己的 64 个后缀 token，外加 2048 前缀中未被缓存覆盖的部分；
2. **Decode 阶段每一个新生成的 token**：每个新 token 都要在前一步基础上重新算 $Q/K/V$ 并追加进缓存，共 max_new_tokens = 16 步；
3. 此外，已命中前缀虽然不必重算，但新算出的 Q （位置 $\geq L$ ）在注意力里**仍要读取**那 L 个位置的 K/V （forward_extend 中做 KV 拼接）。所以复用省掉的是「**算 K/V 并把它们写进缓存**」的算力，而不是省掉对它们的读取。

补充：匹配长度会先按 page_size 对齐（RadixKey.page_aligned）；本实验 page_size = 1，因此命中粒度为单 token。

六、任务一：数据与公式的相互印证

6.1 TTFT 与 TPOT 为什么同时下降——以及不能据此下的结论

先给结论：本次测量能够确证的是 **Prefill 量的大幅下降与端到端时长的下降**；TPOT 的下降**不能**读作「Decode 本身变快了」。

原因在于同一轮的 8 条请求在服务端是**同批同步**推进的，端到端耗时几乎相等（见 6.3 的极差统计）。于是 TTFT 与 TPOT 并非两个彼此独立的观测量，而是**同一段固定时长在「等待」与「解码窗口」之间的分配**：一条请求被准入得越早，它的 TTFT 越小、留给解码的窗口越长，摊到每个输出 token 上的 TPOT 就越大。

同轮「时长守恒」的两个实例（均取 run-1 第一轮 8 条中最先 / 最后被准入的两条）：共享前缀组 $2,039 + 15 \times 1,905 = 17,658 + 15 \times 864 \approx 30,619$ ms；分散前缀组 $12,030 + 15 \times 3,479 \approx 54,606 + 15 \times 641 \approx 64,218$ ms。

从计算量看，Decode 单步与前缀是否命中**无关**：每步仍然是 1 个新 query 对 N 个位置的 K/V，上下文总长 N 与步数（`max_new_tokens = 16`）两组完全一致。前缀缓存改变的是「同一批内还需要做多少 Prefill」，而 Decode 会与这些 Prefill 争抢同一批 CPU。因此跨组比较应以**端到端时长与吞吐**为准，TTFT / TPOT 只作过程量参考。

6.2 端到端差异的定量归因

把每轮 8 条请求看作一个整体：共享前缀组每轮需 Prefill $8 \times 64 = 512$ 个 token，分散前缀组需 $8 \times 2112 = 16,896$ 个，**相差 16,384 个/轮**，四轮合计 65,536 个；而两组每轮的 Decode 步数完全相同（ 8×16 ）、上下文总长也相同。实测两组墙钟之差为 **147.4 s**（ $266.2\text{ s} - 118.9\text{ s}$ ）。

服务端日志里分散组 Prefill 批次自报的吞吐中位数为 **300 tok/s**。若把多出的 65,536 个 token 按这个吞吐串行折算，需要约 218 s；实测只需要 147.4 s，**明显更少**。这是因为 Prefill 与 Decode 在同一批内是重叠推进的，并非严格串行。结论：**端到端差异的量级由多出的 Prefill 量决定**（隐含吞吐 445 tok/s，与服务端自报的 300 tok/s 同一量级），不需要假设 Decode 变快或变慢。

逐项核对：公式预测命中 $L = 2048$ 、每条需 Prefill $N - L = 64$ ，32 条合计 2,048 个 token，实测 2,019 个——略低于预测，是因为少数请求的独立后缀与共享前缀的后续 token 恰好相同，命中长度顺延到了 2049...2054（见 `requests.jsonl` 的 `cached_tokens`）。分散前缀组实测 cached tokens 为 0、实际 Prefill 67,584 个，与 $[L = 0]$ 的预测完全一致。

6.3 并发 8 下的「准入等待」：一个必须说明的实测现象

主表中共享前缀组的 TTFT 达 17,338.0 ms，远高于「已命中 2048 个 token、只剩 65 个 token 需要 prefill」应有的量级。这并不意味着 Prefill 变慢了——服务端日志中本组的前两条 Prefill 记录就是直接证据：

```
[2026-09-29 11:15:16] Prefill batch, #new-seq: 1, #new-token: 64, #cached-token: 2048, token usage: 0.03, #running-req: 0, #queue-req: 0, #pending-token: 0
[2026-09-29 11:15:31] Prefill batch, #new-seq: 7, #new-token: 448, #cached-token: 14336, token usage: 0.03, #running-req: 1, #queue-req: 0, #pending-token: 0
```

首条请求被准入时，调度器的等待队列里**只有它自己**（`#queue-req: 0`）；其余 7 条要再过约 15 s 才**整批**进入（`#new-seq: 7`，即首条之后剩下的 7 条一起进来）。逐请求明细同样指向服务端：把同一轮 8 条请求放在一起看，它们的端到端耗时几乎完全相同（轮内极差最大仅 8.0 ms），说明客户端确实是同时发出的，时间差只能出现在服务端的接收与派发环节。

因此每一批次里多数请求的 TTFT 中包含了一段约 15 s 的**准入等待**，它不是 Prefill 开销。两点判读：

- 这是**纯 CPU 单进程部署**的固有现象：HTTP 接收路径与「调度 + 前向计算」在同一进程内串行，一次长前向执行期间新到的请求无法被及时读取。它与压测脚本、与本次负载设计无关。
- 它对**两组同等存在**（两组每一轮都只有 1 条先被准入），因此两组的**相对差异**仍然可信；而缓存命中率、实际 Prefill token 数等指标直接取自服务端计数器，完全不受时序影响。

另：同一批内多条请求的 Prefill 会被合并进同一个 Prefill batch，服务端日志的批次粒度与「每请求一条」并不一一对应，这也是 4.2 节服务端累计值与客户端 `Σmeta_info` 之间存在小幅差异的原因；核实时以客户端逐请求 `meta_info` 为准（主表）。

6.4 结论

1. **前缀缓存确实生效且收益显著**：共享前缀组把 97.01% 的 prompt token 变成命中，实际 Prefill token 由 67,584 降至 2,019；端到端 p50 由 66,321.2 ms 降至 29,950.0 ms（降幅 54.8%），TTFT p50 降幅 53.8%，输出吞吐由 1.92 升到 4.31 tok/s。
2. **收益来自 Prefill，而不是 Decode 本身**：两组的 Decode 步数（16）与上下文总长完全相同，因此 Decode 单步的计算量没有变化。TPOT p50 的读数从 1,976.4 ms 变为 864.0 ms，属于「同批同步」下的时长再分配（见 6.1），不能据此认为 Decode 变快了。
3. **跨组比较以端到端与吞吐为准**：TTFT / TPOT 在同一轮内此消彼长、二者之和近似恒定，单独引用其中任一都会得到片面的结论（6.1）；而多出的 Prefill 量在多寡上与墙钟之差一致（6.2）。
4. **两组输入输出长度一致、成功率均为 32/32 与 32/32**，说明差异来自前缀能否复用，而不是负载本身的差异。

6.5 执行顺序的稳健性检查（run-2：交换两组顺序）

指标	run-1 (shared → dispersed)	run-2 (dispersed → shared)
共享前缀 缓存命中率	97.01%	97.02%
共享前缀 实际 Prefill tokens	2,019	2,017
共享前缀 TTFT p50 (ms)	17,338.0	15,603.0
分散前缀 TTFT p50 (ms)	37,520.4	39,745.1
分散前缀 实际 Prefill tokens	67,584	67,584

交换顺序后结论不变，说明观测到的差异不是执行顺序（如首组承担额外 JIT / 线程池预热开销）造成的。

七、任务二：一条 /generate 请求的完整流程

一条 /generate 请求在 SGLang v0.5.14 中的处理流程

主流程（任务书指定主线）；括号内为文件路径与行号，均可在 python/sglang/srt 下核对



八、任务二：说明（不超过 1 页）

下图按任务书指定主线绘制，图中 ★ 标记的节点对应本节要回答的四个问题；每个节点下方的等宽文本即该函数所在的文件与行号，均可在 python/sglang/srt 下逐一核对。

8.1 请求如何进入等待队列

HTTP 层 generate_request (entrypoints/http_server.py:769) 在 obj.stream 为真时返回 stream_results() 这个异步生成器。TokenizerManager.generate_request (managers/tokenizer_manager.py:576) 先做参数归一化，再经 _tokenize_one_request (:782) 分词，用 _send_one_request (:1319) 把 TokenizedGenerateReqInput 经 ZMQ 发给 Scheduler 进程。Scheduler 的 event_loop_normal (managers/scheduler.py:1505) 每轮先 recv_requests 取回请求，交给 process_input_requests (:1628)，后者最终调用 _add_request_to_queue (:2258)，把 Req 追加进 self.waiting_queue (:2265) 并打上入队时间戳。到此请求才真正「在等」。

8.2 前缀匹配如何减少本轮 Prefill token

组批时 get_new_batch_prefill (scheduler.py:2702) 先按调度策略 policy.calc_priority (:2761, 默认 cache-aware 的 LPM, 即最长前缀优先) 对等待队列排序，再构造 PrefillAdder (managers/schedule_policy.py:425) 逐条准入。对每条请求，先调用 Req.init_next_round_input (managers/schedule_batch.py:1123)，其中 tree_cache.match_prefix (:1168) 落到 RadixCache.match_prefix (mem_cache/radix_cache.py:358)，由 _match_prefix_helper (:643) 沿基数树逐段比对，返回已命中前缀的 KV slot 索引 prefix_indices (命中中止于节点中部时用 _split_node (:669) 切分节点以暴露精确边界)。随后 set_extend_input_len(len(input) - len(prefix_indices)) (:1221) 把本请求本轮要真正计算的 token 数扣掉已命中的部分。**减少 Prefill token 的机制就是这一步**：这些 token 的 K/V 已在池中被前序请求算好并写入，本轮既不重算也不重写，只需把它们的 slot 索引拼进本轮的 prefix_indices。PrefillAdder.add_one_req (schedule_policy.py:858) 再按剩余 token 预算判定能否纳入本批，通过后组装成 ScheduleBatch (schedule_batch.py:1671)。

8.3 新产生的 KV 如何写回缓存

前向由 Scheduler.run_batch (scheduler.py:3145) 调用 TpModelWorker.forward_batch_generation (managers/tp_worker.py:482) → ModelRunner.forward (model_executor/model_runner.py:2896)，注意力后端按批的 forward_mode 走 forward_extend (Prefill) 或 forward_decode (Decode)。算出的 KV 先按 paged 布局写进 KV pool，并在 req_to_token 里登记 slot。之后由 process_batch_result (scheduler.py:3367) 分派到 batch_result_processor，两条路径回写基数树：**未结束** (chunked prefill 的中间段) 走 maybe_cache_unfinished_req (mem_cache/common.py:113) → RadixCache.cache_unfinished_req (radix_cache.py:485)；**已结束** 走 release_kv_cache (:629) → RadixCache.cache_finished_req (:438) → insert (:418)。插入前序列会按 page_size 做 page_aligned 对齐，再把 token → KV slot 的映射挂到基数树上；重复部分对应的 KV 立即 free 掉，并调整 lock_ref 引用计数，使暂未被引用的节点成为可驱逐 (evictable) 状态。

8.4 生成结果如何流式返回

Decode 的每一步都会经 Scheduler.stream_output (managers/scheduler_components/output_streamer.py:91) 把本步新 token 打包成 BatchTokenIDOutput 发给 Detokenizer 进程；DetokenizerManager.event_loop (managers/detokenizer_manager.py:159) 做增量反分词（只为新 token 解码，避免重复处理整段），再送回 TokenizerManager。_handle_batch_output (tokenizer_manager.py:1839) 组装 meta_info (含 prompt_tokens、completion_tokens、cached_tokens、finish_reason)，_wait_one_response (:1425) 逐片 yield；最终由 http_server.stream_results (http_server.py:778) 以 SSE 形式写出 data: {...}\n\n。客户端读到首个「携带生成 token 的分片」的时刻即 TTFT，之后每片之间的间隔即 TPOT 的实测来源。

与实测的呼应：正因为 8.2 中「已缓存前缀既不重算也不重写」，本实验共享前缀组的实际 Prefill token 才从 67,584 降到 2,019；而 8.4 中 Decode 每步都要重新读入全部已缓存 K/V，且上下文总长 N 与 Decode 步数 (max_new_tokens = 16) 两组完全一致——所以 **Decode 本身的单步计算量并不因前缀命中而减少**，两组 TTFT / TPOT 的变化是「同批同步」下的时长再分配，见 6.1 节。

九、作业感受

(1) 简述下你是如何完成第二次挑战的

第二次挑战复用第一次挑战已经搭好的 SGLang 0.5.14 在线服务 (WSL2 + 纯 CPU + Qwen3-0.6B)，因此第一步不是重建环境，而是**先确认服务的可用状态与缓存配置**，再把注意力放在实验本身的可信度上。

具体过程分三步。第一步是**把任务书翻译成脚本**：把「32 条请求 / 并发 8 / 温度 0 / max_new_tokens 16 / ignore_eos / seed 2026」这些硬性条件写成脚本里的常量，把「组前 flush_cache 并确认成功」「共享前缀组额外发一条预热请求」「预热不计入结果」这些容易漏掉的约束做成流程里的显式步骤，并让每一步都在结果目录留下可核对的文件 (flush_cache.json、warmup_request.json 等)。

第二步是**把控制变量做到位**。任务书要求「除前缀能否复用外其余一致」，我最初的做法是两组各造一份语料，但很快发现这样比出来的差异里混着内容差异。于是改成用「只替换位置 0 的 token」的方式构造分散前缀组：其余 2047 个 token 与共享前缀组逐位相同，并在代码里加了断言把它固定下来。这样唯一的自变量就只剩「前缀能否被 RadixCache 复用」。

第三步是**让数据自己说话**。所有指标都不在报告里手抄，而是由脚本从服务端返回的 meta_info 汇总成 summary.json，再由另一个脚本读它生成报告表格，同时把服务端日志里 Prefill batch 行的 #cached-token 另行汇总作为独立旁证。任务二则反过来：先在源码里逐跳把函数和行号找出来，再用一个生成脚本把它画成流程图，保证图上的每一处标注都能回到源码核对。

AI 在这里的作用主要是**加速定位与校验**：帮我快速在 0.5.14 的源码树里找到 match_prefix、cache_finished_req 等函数的位置，以及提示「命中之后仍需读取已缓存 K/V」这类容易被忽略的细节；但它给出的行号和结论我都回到源码逐条复核过，与 0.5.14 不符的一律以源码为准。

(2) 你觉得哪部分工作最困难，你是如何克服这个困难的

【本题要求「禁止 AI 辅助撰写」，请本人填写】

(3) 类似的从 0 到 1 的科研工作，需要包含哪些方面的研究，对你今后的科研工作有什么启发

【本题要求「禁止 AI 辅助撰写」，请本人填写】